Exercise 6 – Evaluation II

Introduction to Machine Learning

Hint: Useful libraries

R

```
# Consider the following libraries for this exercise sheet:

library(mlbench)
library(mlr3)
library(mlr3learners)
```

Python

```
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd

# sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.model_selection import RepeatedKFold
from sklearn.model_selection import RepeatedStratifiedKFold
```

Exercise 1: Overfitting & underfitting

Learning goals

1. Discuss for given situations if they might lead to overfitting / underfitting
2. Judge whether over- and underfitting are properties of a learner or of a fixed model
3. Discuss to what extent over- and underfitting can be avoided

Assume a polynomial regression model with a continuous target variable y, a continuous, p-dimensional feature vector $\mathbf{x}$ and polynomials of degree d, i.e.,

$$f\left(\mathbf{x}^{(i)}\right) = \theta_0 + \sum_{j=1}^{p} \sum_{k=1}^{d} \theta_{j,k} (\mathbf{x}_j^{(i)})^k.$$

a) For each of the following situations, indicate whether we would generally expect the performance of a flexible polynomial learner (high d) to be better or worse than an inflexible one (low d). Justify your answer.

NB

We can only state tendencies here; performance strongly depends on the specific data situation.

i. The sample size n is extremely large, and the number of features p is small.

Solution

We expect the flexible learner to perform better because it covers a large number of hypotheses and we have enough data to tell them apart, so there is little risk of overfitting, while the inflexible learner might underfit.

ii. The number of features p is extremely large, and the number of observations n is small.

Solution

With a flexible learner we might quickly run into overfitting here. The data situation, which we frequently encounter in biostatistics (e.g., genomics data), creates a high-dimensional and sparsely populated input space with few training points (relatively speaking) that are easy to overfit. Therefore, a low-degree polynomial should fare better (but note that it is not immune to overfitting either in this challenging setting). Some sort of feature selection could be advisable first.

iii. The true relationship between the features and the response is highly non-linear.

Solution

The inflexible learner will likely underfit here, so the flexible variant can be expected to perform better.

iv. The data could only be observed with a high level of noise.

Solution

The flexible learner might interpolate between the noisy training samples, creating a wiggly curve that generalizes poorly, so we expect better performance from a less complex learner.

b) Are overfitting and underfitting properties of a learner or of a fixed model? Explain your answer.

Solution

Overfitting and underfitting are always connected to a particular fixed *model*, even though attributes of the underlying hypothesis space typically influence the tendency toward one or the other behavior, as we have seen in the previous question. In order to understand this, think of a classification problem with linearly separable data. Applying a QDA learner, which is able to learn more complex decision boundaries, poses a risk of overfitting with the chosen model, but the degree of overfitting depends on the model itself. In theory, the QDA learner is free to set equal covariances for the Gaussian class densities, amounting to LDA and *not* overfitting the data. Under- and overfitting are therefore properties of a specific model and not of an entire learner.

A common strategy is to choose a rather flexible model class and encourage simplicity in the actual model by *regularization* (e.g., take a higher-degree polynomial but drive as many coefficients a possible toward zero, which you might know as LASSO regression).

c) Should we aim to completely avoid both overfitting and underfitting?

Solution

That will be hardly possible. Recall how we defined the two variants of data fit:

$$UF(\hat{f}, L) = GE(\hat{f}, L) - GE(f^*, L)$$

$$OF(\hat{f}, L) = GE(\hat{f}, L) - \mathcal{R}_{\text{emp}}(\hat{f}, L)$$

In order to avoid underfitting completely we would need to always find the universally loss-optimal model across arbitrary hypothesis spaces (the so-called *Bayes-optimal* model),

which is obviously not something we can hope to achieve in general. Zero overfitting would mean to exactly balance theoretical generalization error and empirical risk, but the way empirical risk minimization is designed, our model will likely fare a bit worse on unseen test data.

In practice we will always experience these phenomena to some degree and finding a model that trades them off well is the holy grail in machine learning.

Exercise 2: Resampling in practice

Learning goals

1. Implement resampling procedures in R/Python
2. Understand how the choice of resampling strategy affects the quality of the GE estimator

a) Why would we apply resampling rather than a single holdout split?

Solution

The two main advantages of resampling are:

- We are able to use larger training sets (at the expense of test set size) because the high variance this incurs for the resulting estimator is smoothed out by averaging across repetitions.
- Repeated sampling reduces the risk of getting lucky (or not so lucky) with a particular data split, which is especially relevant with few observations.

b) Classify the `german_credit` data into solvent and insolvent debtors using logistic regression. Compute the training error w.r.t. MCE.

Python Hint

Read the already preprocessed file german_credit_for_py.csv

Solution

R

```r
# create task and learner
(task <- tsk("german_credit"))
learner <- lrn("classif.log_reg")

# train, predict and compute train error
learner$train(task)
preds <- learner$predict(task)
preds$score()
```

```
<TaskClassif:german_credit> (1000 x 21): German Credit
* Target: credit_risk
* Properties: twoclass
* Features (20):
  - fct (14): credit_history, employment_duration, foreign_worker,
    housing, job, other_debtors, other_installment_plans,
    people_liable, personal_status_sex, property, purpose, savings,
    status, telephone
  - int (3): age, amount, duration
  - ord (3): installment_rate, number_credits, present_residence
```

classif.ce: 0.211

Python

Read data

```python
german_credit = pd.read_csv("../data/german_credit_for_py.csv")

# preview: the categorical features come already one-hot-encoded
print(german_credit.shape)
print(german_credit.head()[["credit_risk", "duration", "amount", "age"]])
```

```
(1000, 49)
  credit_risk  duration  amount  age
0        good         6    1169   67
1         bad        48    5951   22
2        good        12    2096   49
3        good        42    7882   45
4         bad        24    4870   53
```

Encode data and train classifier

```
german_x_raw = german_credit.iloc[:, 1:]
german_y_raw = german_credit.iloc[:, 0]

# Initialize encoder for target
enc_target = LabelEncoder()

enc_target.fit(german_y_raw.values.ravel())
# .values will give the values in a numpy array (shape: (n,1))
# .ravel will convert that array shape to (n, ) (i.e. flatten it)

# now numpy array
german_y = enc_target.transform(german_y_raw.values.ravel())
# you can also use enc_target.fit_transform(X) to combine both steps

german_x = np.asarray(german_x_raw)

# Using whole data set to train and predict;
# increase max iterations for convergence
log_mod = LogisticRegression(max_iter=10000).fit(german_x, german_y)

acc = log_mod.score(german_x, german_y)
print("Mean accuracy: %.2f" % acc)
print("Mean classification error: %.2f" % (1 - acc))
```

```
Mean accuracy: 0.79
Mean classification error: 0.21
```

c) In order to evaluate your learner, compare the test MCE using

1. three times ten-fold cross validation (3x10-CV)
2. 10x3-CV
3. 3x10-CV with stratification for the feature `foreign_worker` to ensure equal representation in all folds
4. a single holdout split with 90% training data

Hint

R

You will need `rsmp`, `resample` and `aggregate`.

Python

You will need `RepeatedKFold`, `RepeatedStratifiedKFold` and `train_test_split`.

Solution

R

Define the resampling strategies and evaluate the learner:

```
# create different resampling strategies
set.seed(123)
resampling_3x10_cv <- rsmp("repeated_cv", folds = 10, repeats = 3)
resampling_10x3_cv <- rsmp("repeated_cv", folds = 3, repeats = 10)
resampling_ho <- rsmp("holdout", ratio = 0.9)

# evaluate without stratification
result_3x10_cv <- resample(
  task,
  learner,
  resampling_3x10_cv,
  store_models = TRUE
)
result_10x3_cv <- resample(
  task,
  learner,
  resampling_10x3_cv,
  store_models = TRUE
)
result_ho <- resample(task, learner, resampling_ho, store_models = TRUE)

# evaluate with stratification
task_stratified <- task$clone()
task_stratified$set_col_roles("foreign_worker", roles = "stratum")
result_stratified <- resample(
  task_stratified,
  learner,
  resampling_3x10_cv,
  store_models = TRUE
)
```

Aggregate the test errors over the splits (MCE by default):

```
# aggregate results over splits (mce is default)
print(sapply(
  list(result_3x10_cv, result_10x3_cv, result_stratified, result_ho),
  function(i) i$aggregate()
))
```

```
classif.ce classif.ce classif.ce classif.ce
 0.2486667  0.2557977  0.2525512  0.1800000
```

Python

(i) 3x10-CV:

```
random_state = 14
err = []
rkf_3x10 = RepeatedKFold(n_splits=10, n_repeats=3, random_state=random_state)
for train, test in rkf_3x10.split(german_x):
    log_mod = LogisticRegression(max_iter=10000).fit(
        german_x[train, :], german_y[train]
    )
    err.append(1 - log_mod.score(german_x[test, :], german_y[test]))
    # score gives mean accuracy

res = np.array(err)
print("MCE of 3x10 CV: ", res.mean())
```

```
MCE of 3x10 CV:  0.24366666666666664
```

(ii) 10x3-CV:

```
err = []
rkf_10x3 = RepeatedKFold(n_splits=3, n_repeats=10, random_state=random_state)
for train, test in rkf_10x3.split(german_x):
    log_mod = LogisticRegression(max_iter=10000).fit(
        german_x[train, :], german_y[train]
    )
    err.append(1 - log_mod.score(german_x[test, :], german_y[test]))

res = np.array(err)
print("MCE of 10x3 CV: ", res.mean())
```

```
MCE of 10x3 CV:  0.2519924115732499
```

(iii) 3x10-CV with stratification for `foreign_worker`:

```
err = []
strat_gkf_10 = RepeatedStratifiedKFold(
    n_splits=10, n_repeats=3, random_state=random_state
)
# Note that providing y in split(X, y) is sufficient to generate the splits,
# and hence np.zeros(n_samples) may be used as a placeholder for X instead
# of actual training data.
for train, test in strat_gkf_10.split(german_x, german_x[:, 41]):
    # index 41 stands for column of foreign_workers_yes
    log_mod = LogisticRegression(max_iter=10000).fit(
        german_x[train, :], german_y[train]
    )
    err.append(1 - log_mod.score(german_x[test, :], german_y[test]))

res = np.array(err)
print("MCE of 3x10-CV with stratification: ", res.mean())
```

```
MCE of 3x10-CV with stratification:  0.2486666666666667
```

(iv) Holdout with 90% training data:

```
x_train, x_test, y_train, y_test = train_test_split(
    german_x, german_y, test_size=0.1, random_state=random_state
)
log_mod = LogisticRegression(max_iter=10000).fit(x_train, y_train)
german_pred = log_mod.predict(x_test)
print("MCE of Hold-out split: ", 1 - log_mod.score(x_test, y_test))
```

```
MCE of Hold-out split:  0.32999999999999996
```

d) Discuss and compare your findings and compare them to the training error computed previously.

Solution

Generalization error estimates are pretty stable across the different resampling strategies because we have a fairly large number (1000) of observations. Still, the pessimistic bias of small training sets is visible: 10x3-CV, using roughly 67% of data for training in each split, estimates a higher generalization error than 3x10-CV with roughly 90% training data. Stratification by `foreign_worker` does not seem to have much effect on the estimate. However, we see a glaring difference when we use a single 90%-10% split, where the estimated GE is quite different from 3x10-CV, meaning we got a different error just because of an (un)lucky split.

Comparing the results (except for the unreliable one produced by a single split) with the training error from before indicates no serious overfitting.

Exercise 3: Computational cost of resampling

The whole exercise is only for lecture group B!

Learning goals

1. Determine how many models a resampling strategy fits
2. Reason about the computational cost of different resampling strategies
3. Relate computational cost to the quality of the GE estimate

Recall that a resampling strategy is a collection of B train/test splits,

$$\mathcal{J} = \left((J_{\text{train},1}, J_{\text{test},1}), \dots, (J_{\text{train},B}, J_{\text{test},B})\right),$$

and that the learner is refitted on each of the B training sets.

a) A data set has $n = 200$ observations. State how many models are fitted for each of the following strategies:

   i. 10-fold cross-validation (10-CV)
   ii. 5-times repeated 10-fold cross-validation (5x10-CV)
   iii. leave-one-out CV (LOO-CV)
   iv. subsampling with 25 iterations

Hint

The number of model fits equals the number of splits B.

Solution

Each split requires exactly one model fit, so we simply count the splits:

   i. 10-CV: 10 models.
   ii. 5x10-CV: $5 \cdot 10 = 50$ models.
   iii. LOO-CV: one model per observation, i.e. $n = 200$ models.
   iv. Subsampling with 25 iterations: 25 models.

b) Now give the number of model fits **in general**, i.e. as a formula, for a data set of size n with k folds, r repetitions and B iterations:

   i. a single holdout split
   ii. k-fold CV
   iii. r-times repeated k-fold CV
   iv. LOO-CV

v. subsampling or bootstrap with B iterations

Solution

i. holdout: 1
ii. k-fold CV: k
iii. r-times repeated k-fold CV: $r \cdot k$
iv. LOO-CV: n (the special case $k = n$)
v. subsampling / bootstrap with B iterations: B

c) Explain why LOO-CV becomes computationally prohibitive for large n, and name a resampling strategy that achieves a similarly low (pessimistic) bias at much lower cost.

Solution

LOO-CV fits exactly n models, so its cost grows linearly with the sample size. For large n (say 10^5 to 10^6) this becomes infeasible – and LOO additionally suffers from high variance. Repeated k-fold CV with a moderate k (e.g. 5 or 10) trains on almost as much data (hence a similarly small pessimistic bias) but fits only $r \cdot k \ll n$ models, while averaging over the repetitions keeps the variance low. It is therefore usually the better choice.

d) Both 3-times repeated 10-fold CV (3x10-CV) and 10-times repeated 3-fold CV (10x3-CV) fit exactly 30 models, so their computational cost is roughly the same. Does this mean that both strategies provide GE estimates of the same quality? Explain your answer.

Solution

No. The number of model fits only measures the computational cost, not the quality of the resulting estimate. With $k = 10$ folds, each model is trained on 90% of the data, while $k = 3$ uses only about 67%. Models trained on considerably less data tend to perform worse, so 10x3-CV overestimates the generalization error of the final model (which is trained on the full data set) more strongly, i.e., it has a larger pessimistic bias. This matches what we observed empirically in Exercise 2 d), where 10x3-CV produced a higher error estimate than 3x10-CV.

(As a side note, the costs are not even exactly equal: training on 67% of the data is usually somewhat faster than training on 90%.)

Exercise 4: Variance and independence of GE estimators

The whole exercise is only for lecture group A!

Learning goals

1. Derive mean and variance of the holdout GE estimator on a fresh i.i.d. test set
2. Understand why the per-fold errors of cross-validation are not independent

Consider a fixed model $\hat{f}$ that has been fitted on some training data and is now held fixed. We estimate its generalization error on a test set $\mathcal{D}_{\text{test}}$ of n_{test} observations drawn i.i.d. from $\mathbb{P}_{xy}$, independently of the training data:

$$\widehat{\text{GE}}\left(\hat{f}\right) = \frac{1}{n_{\text{test}}} \sum_{(\mathbf{x},y)\in\mathcal{D}_{\text{test}}} L\left(y, \hat{f}(\mathbf{x})\right).$$

a) Show that $\widehat{\text{GE}}\left(\hat{f}\right)$ is an unbiased estimator of the theoretical generalization error $\text{GE}\left(\hat{f}\right) = \mathbb{E}\left[L\left(y, \hat{f}(\mathbf{x})\right)\right]$ of the fixed model $\hat{f}$.

Solution

Write $L^{(i)} := L\left(y^{(i)}, \hat{f}\left(\mathbf{x}^{(i)}\right)\right)$ for the n_{test} test observations. Because $\hat{f}$ is fixed and the test points are i.i.d. $\sim \mathbb{P}_{xy}$, the $L^{(i)}$ are i.i.d. with common mean $\mathbb{E}\left[L^{(i)}\right] = \mathbb{E}\left[L\left(y, \hat{f}(\mathbf{x})\right)\right] = \text{GE}\left(\hat{f}\right)$. By linearity of expectation,

$$\mathbb{E}\left[\widehat{\text{GE}}\left(\hat{f}\right)\right] = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{E}\left[L^{(i)}\right] = \frac{1}{n_{\text{test}}} \cdot n_{\text{test}} \cdot \text{GE}\left(\hat{f}\right) = \text{GE}\left(\hat{f}\right).$$

Hence the test-set estimator is unbiased for the GE of the fixed model.

b) Show that $\mathsf{Var}\left[\widehat{\text{GE}}\left(\hat{f}\right)\right] = \frac{1}{n_{\text{test}}}\mathsf{Var}\left[L\left(y, \hat{f}(\mathbf{x})\right)\right]$ and interpret how the estimator's precision depends on the test set size.

Solution

Since the $L^{(i)}$ are i.i.d., set $\sigma^2 := \mathsf{Var}\left[L\left(y, \hat{f}(\mathbf{x})\right)\right]$. Using $\mathsf{Var}[aX] = a^2\mathsf{Var}[X]$ and independence (so variances add),

$$\mathsf{Var}\left[\widehat{\mathrm{GE}}\left(\hat{f}\right)\right] = \mathsf{Var}\left(\frac{1}{n_{\text{test}}}\sum_{i=1}^{n_{\text{test}}} L^{(i)}\right) = \frac{1}{n_{\text{test}}^2}\sum_{i=1}^{n_{\text{test}}} \mathsf{Var}\left[L^{(i)}\right] = \frac{1}{n_{\text{test}}^2}\cdot n_{\text{test}}\,\sigma^2 = \frac{\sigma^2}{n_{\text{test}}}.$$

The variance decreases in inverse proportion to the test set size: a larger test set yields a more precise (lower-variance) estimate. This is exactly why a single holdout split with a small test set produces a high-variance GE estimate.

c) In k-fold cross-validation we obtain k per-fold estimates $\widehat{\mathrm{GE}}^{(1)}, \ldots, \widehat{\mathrm{GE}}^{(k)}$ and report their mean. One might be tempted to treat these k numbers as k independent measurements – e.g., to report their standard deviation or run a t-test to compare two learners. Explain why the $\widehat{\mathrm{GE}}^{(j)}$ are **not** independent, and what this implies for such a variance estimate.

Solution

The k training sets overlap strongly – any two of them share $k-2$ of the k folds – so the models fitted across folds are correlated. In addition, every observation serves as a training point in $k-1$ folds and as a test point in exactly one, which further couples the per-fold errors. The $\widehat{\mathrm{GE}}^{(j)}$ are therefore (positively) dependent, not independent.

Consequently, treating them as i.i.d. and using their empirical variance $\frac{1}{k}\sum_{j=1}^{k}\left(\widehat{\mathrm{GE}}^{(j)} - \overline{\widehat{\mathrm{GE}}}\right)^2$ **underestimates** the true variance of the CV estimator, and a t-test based on this assumption is anti-conservative (it flags too many differences as significant). In fact, there is no unbiased estimator of the variance of the k-fold CV estimator [Bengio & Grandvalet, 2004], so CV results should be used with care for formal significance testing.